TALLER SEMANA 4

IMPLEMENTACIÓN PATRON ABSTRACT FACTORY

DOCENTE

ELIECER MONTERO OJEDA

INTEGRANTE

JUAN DAVID RENGIFO CHAVEZ

UNIDADES TECONOLOGICAS DE SANTANDER
PATRONES DE SOFTWARE
INGENIERIA EN SISTEMAS
2026

1. Estructura del módulo Energía (Abstract Factory)

```
└── 📁energia
    └── 📁aplicacion
        └── 📁servicio
            ├── EnergyOfferService.java
    └── 📁dominio
        └── 📁factory
            ├── AuctionSaleFactory.java
            ├── DirectSaleFactory.java
            ├── EnergySaleFactory.java
        └── 📁modelo
            ├── Auction.java
            ├── EnergyOffer.java
            ├── SaleType.java
            ├── Transaction.java
        └── 📁proceso
            ├── AuctionSaleProcess.java
            ├── DirectSaleProcess.java
            ├── SaleProcess.java
        └── 📁puerto
            ├── EnergyOfferRepositoryPort.java
    └── 📁infraestructura
        └── 📁configuracion
            ├── EnergySaleFactoryConfig.java
        └── 📁controlador
            ├── EnergyOfferController.java
            ├── EnergyOfferRequest.java
        └── 📁persistencia
            ├── EnergyOfferEntity.java
            ├── EnergyOfferJpaRepository.java
            └── EnergyOfferRepositoryAdapter.java
```

En este taller se trabaja sobre el módulo energia del proyecto Enerlink, donde se implementa el patrón Abstract Factory para manejar dos familias de productos relacionados con la venta de energía: venta directa y subasta.

- El paquete dominio.factory define la interfaz abstracta EnergySaleFactory y sus fábricas concretas DirectSaleFactory y AuctionSaleFactory.

- El paquete dominio.proceso define la familia de procesos de venta (SaleProcess) con sus variantes DirectSaleProcess y AuctionSaleProcess.

- El servicio de aplicación EnergyOfferService usa estas fábricas para crear ofertas y ejecutar el proceso correcto sin conocer las clases concretas.

Esta organización respeta la Arquitectura Hexagonal, ya que las fábricas pertenecen al dominio y son orquestadas desde la capa de aplicación, mientras la infraestructura solo configura e inyecta dependencias.

2. Patrón Abstract Factory

2.1 Descripción del patrón

El Abstract Factory es un patrón de diseño creacional que define un contrato para crear familias de objetos relacionados, garantizando que las combinaciones de productos sean coherentes entre sí sin acoplarse a clases concretas.

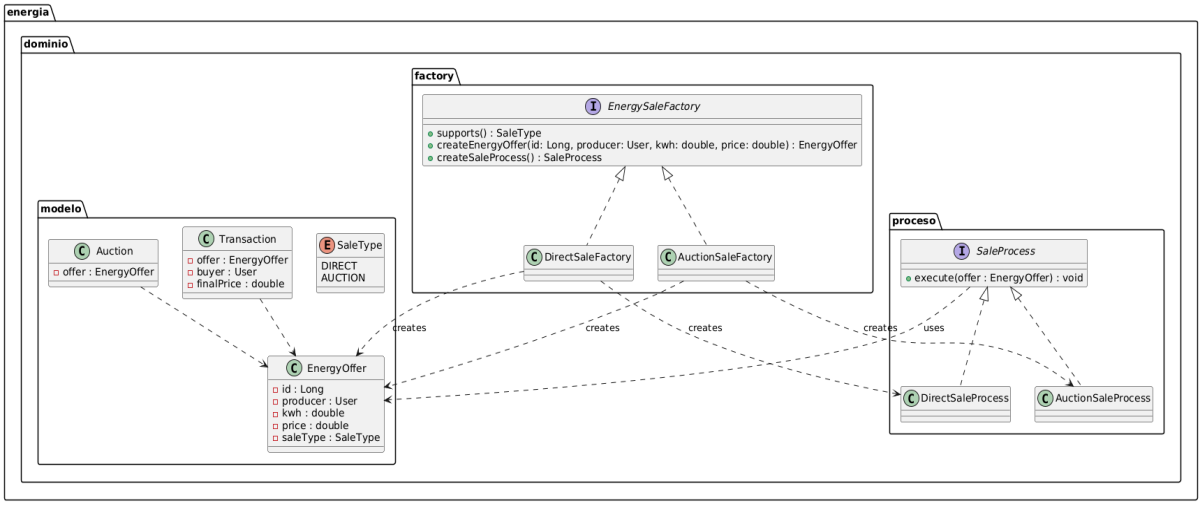
En Enerlink, este patrón se utiliza para que la capa de aplicación pueda crear ofertas de energía y procesos de venta consistentes (venta directa o subasta) sin depender directamente de DirectSaleProcess, AuctionSaleProcess ni de constructores concretos de EnergyOffer.
Desde el punto de vista de diseño, el patrón favorece:

- Open/Closed Principle: nuevos tipos de venta (por ejemplo, venta programada) se pueden agregar creando nuevas fábricas concretas sin modificar el código existente del servicio que orquesta las ventas.

- Single Responsibility Principle: cada fábrica concreta (DirectSaleFactory, AuctionSaleFactory) se encarga únicamente de crear la familia de objetos correspondiente a su tipo de venta.

- Dependency Inversion Principle: la capa de aplicación depende de la abstracción EnergySaleFactory y del contrato SaleProcess, en lugar de acoplarse a clases concretas, lo que facilita pruebas, mantenimiento y extensibilidad.

2.2 Diagrama UML



2.3 Implementación

Clase EnergyOffer

```java
package com.enerlink.enerlink.energia.dominio.modelo;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class EnergyOffer {

    private Long id;
    private User producer;
    private double kwh;
    private double price;
    private SaleType saleType;

    public EnergyOffer(Long id, User producer, double kwh, double price, SaleType saleType) {
        this.id = id;
        this.producer = producer;
        this.kwh = kwh;
        this.price = price;
        this.saleType = saleType;
    }

```

EnergyOffer representa una oferta energética publicada por un productor, indicando cantidad en kWh, precio y tipo de venta (DIRECT o AUCTION).

Clase enum SaleType

```java
package com.enerlink.enerlink.energia.dominio.modelo;

public enum SaleType {
    DIRECT,
    AUCTION
}
```

SaleType define las variantes soportadas por el sistema: venta directa y subasta.

Clase Auction

```java
package com.enerlink.enerlink.energia.dominio.modelo;

public class Auction {

    private EnergyOffer offer;

    public Auction(EnergyOffer offer) {
        this.offer = offer;
    }
}

```

Auction modela la subasta asociada a una oferta configurada como subasta.

Clase Transaction

```java
package com.enerlink.enerlink.energia.dominio.modelo;

import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class Transaction {

    private EnergyOffer offer;
    private User buyer;
    private double finalPrice;

    public Transaction(EnergyOffer offer, User buyer, double finalPrice) {
        this.offer = offer;
        this.buyer = buyer;
        this.finalPrice = finalPrice;
    }
}

```

Transaction representa la compra final de una oferta, ya sea por venta directa o por subasta.

Interfaz de la fábrica abstracta

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public interface EnergySaleFactory {

    SaleType supports();

    EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price);

    SaleProcess createSaleProcess();

}

```

- supports() indica para qué tipo de venta es responsable la fábrica concreta.
- createEnergyOffer(...) crea una oferta coherente con el tipo de venta de la fábrica.
- createSaleProcess() devuelve el proceso de venta apropiado (DirectSaleProcess o AuctionSaleProcess).

Fabrica concreta DirectSaleFactory

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.DirectSaleProcess;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class DirectSaleFactory implements EnergySaleFactory {

    @Override
    public SaleType supports() {
        return SaleType.DIRECT;
    }

    @Override
    public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
        return new EnergyOffer(id, producer, kwh, price, SaleType.DIRECT);
    }

    @Override
    public SaleProcess createSaleProcess() {
        return new DirectSaleProcess();
    }
}

```

DirectSaleFactory produce una EnergyOffer configurada como venta directa y un DirectSaleProcess para ejecutar la compra inmediata.

Fabrica concreta AuctionSaleFactory

```java
package com.enerlink.enerlink.energia.dominio.factory;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;
import com.enerlink.enerlink.energia.dominio.modelo.SaleType;
import com.enerlink.enerlink.energia.dominio.proceso.AuctionSaleProcess;
import com.enerlink.enerlink.energia.dominio.proceso.SaleProcess;
import com.enerlink.enerlink.usuario.dominio.modelo.User;

public class AuctionSaleFactory implements EnergySaleFactory {

    @Override
    public SaleType supports() {
        return SaleType.AUCTION;
    }

    @Override
    public EnergyOffer createEnergyOffer(Long id, User producer, double kwh, double price) {
        return new EnergyOffer(id, producer, kwh, price, SaleType.AUCTION);
    }

    @Override
    public SaleProcess createSaleProcess() {
        return new AuctionSaleProcess();
    }
}
```

AuctionSaleFactory produce una EnergyOffer del tipo subasta y un AuctionSaleProcess que encapsula la lógica de subasta.

Interfaz SalesProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public interface SaleProcess {

    void execute(EnergyOffer offer);

}
```

Define el contrato común para ejecutar un proceso de venta sobre una oferta de energía.

Implementación DirectSaleProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public class DirectSaleProcess implements SaleProcess {

    @Override
    public void execute(EnergyOffer offer) {
        // lógica de venta directa
        // aquí se crearía la Transaction
        System.out.println("Ejecutando venta directa...");
    }
}

```

DirectSaleProcess encapsula la lógica de compra directa, donde la oferta se vende inmediatamente generando una transacción.

Implementación AuctionSaleProcess

```java
package com.enerlink.enerlink.energia.dominio.proceso;

import com.enerlink.enerlink.energia.dominio.modelo.EnergyOffer;

public class AuctionSaleProcess implements SaleProcess {

    @Override
    public void execute(EnergyOffer offer) {
        // lógica de subasta
        // aquí se crearía la Auction
        System.out.println("Ejecutando subasta...");
    }
}

```

AuctionSaleProcess encapsula la lógica de subasta, en la que se gestionan pujas y se crea una Auction asociada a la oferta.

3. Justificación de la implementación

Técnicamente:

- La capa de aplicación nunca hace new DirectSaleProcess(...) ni new AuctionSaleProcess(...), ni construye directamente EnergyOffer acoplada a un SaleType concreto.

- La creación de ofertas y procesos de venta se delega a la fábrica correspondiente según el tipo de venta (DIRECT, AUCTION), garantizando que el servicio no dependa de clases concretas, sino de EnergySaleFactory y SaleProcess.

- Si se requiere un nuevo tipo de venta, solo hay que agregar una nueva fábrica concreta (por ejemplo, ScheduledSaleFactory) e integrarla en la configuración, sin modificar la lógica central del servicio de aplicación.

Desde la perspectiva del negocio en Enerlink:

- Cada tipo de venta puede tener un comportamiento propio (por ejemplo, lógica específica para venta directa o reglas adicionales para subastas), encapsulado en su implementación de SaleProcess.

- La fábrica asegura que, para cada tipo de venta, se construya siempre una combinación coherente de EnergyOffer y SaleProcess, garantizando consistencia en la forma en que se gestionan las ofertas energéticas.

- Se reducen errores al centralizar la lógica de creación por tipo de venta y se mejora la capacidad de escalar el sistema a nuevos mecanismos de intercambio energético.

Relación con Clean Architecture y SOLID:

- La capa de aplicación depende de abstracciones (EnergySaleFactory, SaleProcess) y no de clases concretas, aplicando el principio de Inversión de Dependencias.

- Cada fábrica concreta tiene una única responsabilidad: crear la familia de objetos correspondiente a un tipo de venta específico (oferta + proceso de venta), respetando el Single Responsibility Principle.

- El sistema es abierto a extensiones y cerrado a modificaciones, ya que nuevos tipos de venta se añaden mediante nuevas fábricas sin cambiar el código existente de los servicios, cumpliendo el Open/Closed Principle.

4. Aclaración sobre la implementación actual

Actualmente, las fábricas de venta de energía se enfocan en construir objetos simples para demostrar correctamente el patrón Abstract Factory dentro del contexto de Enerlink, sin incorporar todavía reglas avanzadas de negocio ni cálculos complejos asociados a la transacción o la subasta.

Esta implementación mínima permite validar el diseño del patrón y su integración con la arquitectura hexagonal sin añadir lógica extra, facilitando la comprensión del rol de cada componente. A futuro, si se requiere mejorar el flujo de creación (por ejemplo, inicializar estados específicos de la subasta, aplicar comisiones, configurar parámetros adicionales de la transacción o asociar políticas de validación por tipo de venta), estas responsabilidades podrán incorporarse dentro de las fábricas concretas y de los procesos de venta correspondientes, manteniendo intacta la interfaz EnergySaleFactory y sin romper la estructura actual del patrón.

Referencias Bibliográficas

Refactoring.Guru. (s. f.). Abstract factory. Refactoring.Guru. https://refactoring.guru/design-patterns/abstract-factory

Refactoring.Guru. (s. f.). Creational patterns. Refactoring.Guru. https://refactoring.guru/design-patterns/creational-patterns

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.

Freeman, E., Freeman, E., Sierra, K., & Bates, B. (2004). Head first design patterns. O'Reilly Media.

Martin, R. C. (2018). Clean architecture: A craftsman's guide to software structure and design. Prentice Hall.

Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.